SPRING MVC



UP ASI
Bureau E204

Plan du Cours

- Spring MVC (Définition + Spring web)
- Serveur web vs. Serveur d'application
- Les architectures physiques et logiques
- Spring MVC
- Postman
- Dépendance Web
- Cycle de Vie d'une requête HTTP (Spring Boot + Postman)
- @Service + @RestController

– TP Spring Boot + Spring Data JPA + Spring MVC (REST) + Postman

Introduction

- Un **Conteneur de Servlets** (Servlet container en anglais) ou **Conteneur Web** (web container en anglais) est un logiciel qui exécute des servlets.

- Un ou une **Servlet** est une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP.

- Il existe plusieurs conteneurs de servlets, dont **Apache Tomcat** ou encore Jetty. Le serveur d'application JBoss Application Server(Wildfly) utilise Apache Tomcat.

- Nous allons nous intéresser au développement de la couche **Web** (Web Services REST + Contrôleur + Service + Repository) dans ce cours.
- Nous allons aussi pratiquer la consommation des services par Postman.

Spring WEB

- Plusieurs Projets Spring permettent d'implémenter des applications Web :

- Framework Spring (qui contient Spring MVC)
- Spring Web Flow (Implémenter les navigations Stateful).
- Spring mobile (Détecter le type de l'appareil connecté).
- Spring Social (Facebook, Twitter, LinkedIn).
- …

- Nous allons nous intéresser à **Spring MVC**.

SPRING MVC

- **Spring MVC** est un Framework Web basé sur le design pattern **MVC** (Model / View / Controller).
- Spring MVC fait partie du projet “Spring Framework”.
- Spring MVC s'intègre avec les différentes technologies de vue tel que JSF, JSP, Velocity, Thymeleaf...
- Spring MVC n’offre pas une technologie de vue mais permet en revanche de communiquer avec toutes les technologies web les plus performantes telles que Angular, React, etc...
- Spring MVC est construit en se basant sur la spécification JavaEE : **Java Servlet**.

Architecture Physique

- **Tier** est un mot anglais qui signifie étage ou niveau.
- Une application peut être **1-Tier**, **2-Tiers**, **3-Tiers** ou **N-Tiers**.

Architecture Physique - 1-Tiers

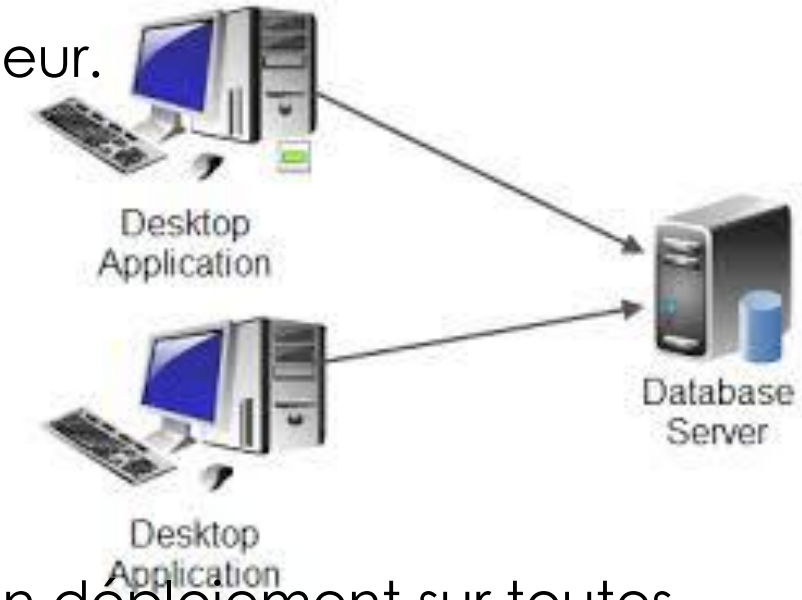
- Une application **1-Tier** est, par exemple, la Modification d'un document Word sur un ordinateur Local.
- Tout est sur la même machine et les couches sont fortement liées.
- Inconvénients : Risque de perte des données (non sauvegardées à distance), Impossible d'accéder à une même ressource par deux utilisateurs en même temps.



Architecture Physique - 2-Tiers

- Une application **2-Tiers** est typiquement une application **client lourd**.

- Le niveau **Présentation (IHM)** et le niveau **Traitement** sont sur la machine de l'utilisateur.
- Le niveau **Base de Données** est sur un autre serveur.



- C'est une architecture **Client / Serveur**.
- Client = demandeur de ressource
- Serveur = fournisseur de ressource

Inconvénients

- Toute mise à jour des fonctionnalités nécessite un déploiement sur toutes les machines des utilisateurs.
- Le serveur ne fait pas appel à une autre application pour fournir le service.

Architecture Physique - 3-Tiers

- Une application **3-Tiers** introduit un niveau intermédiaire (middleware) entre le client et le serveur.
- Le niveau intermédiaire est chargé de fournir la ressource en faisant appel à un autre serveur.

Avantages

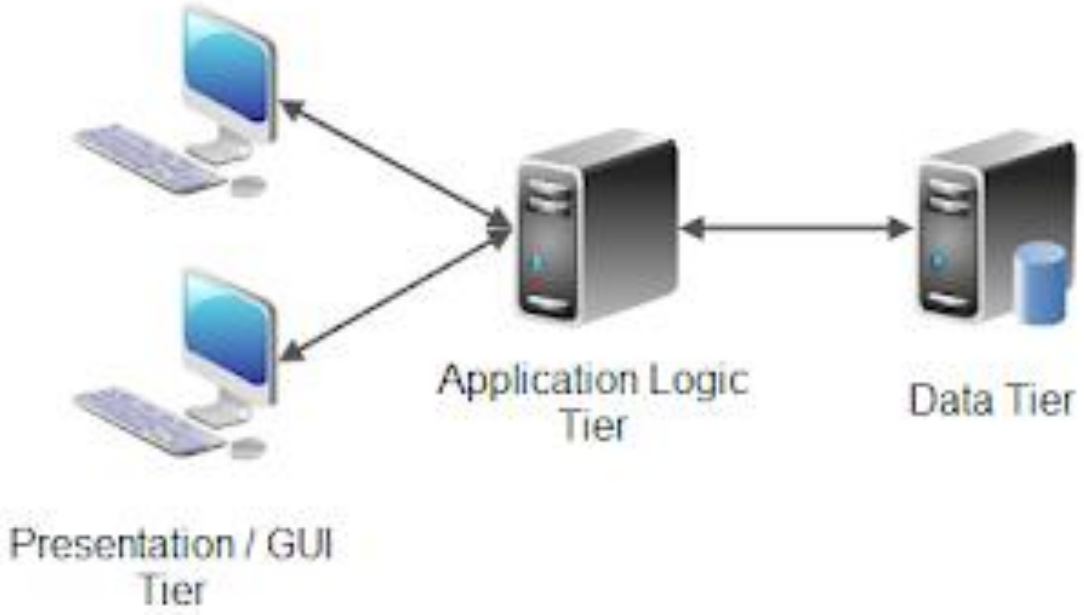
- Centraliser la logique application sur un serveur HTTP

Inconvénients

- Le serveur HTTP (élément principal de l'architecture)est fortement sollicité d'où une charge de demandes provenant à la fois du client et du serveur.
- Bien que cette architecture résout le problème du client lourd de l'architecture deux tiers, le soulagement du client est remplacé par un serveur fortement sollicité.

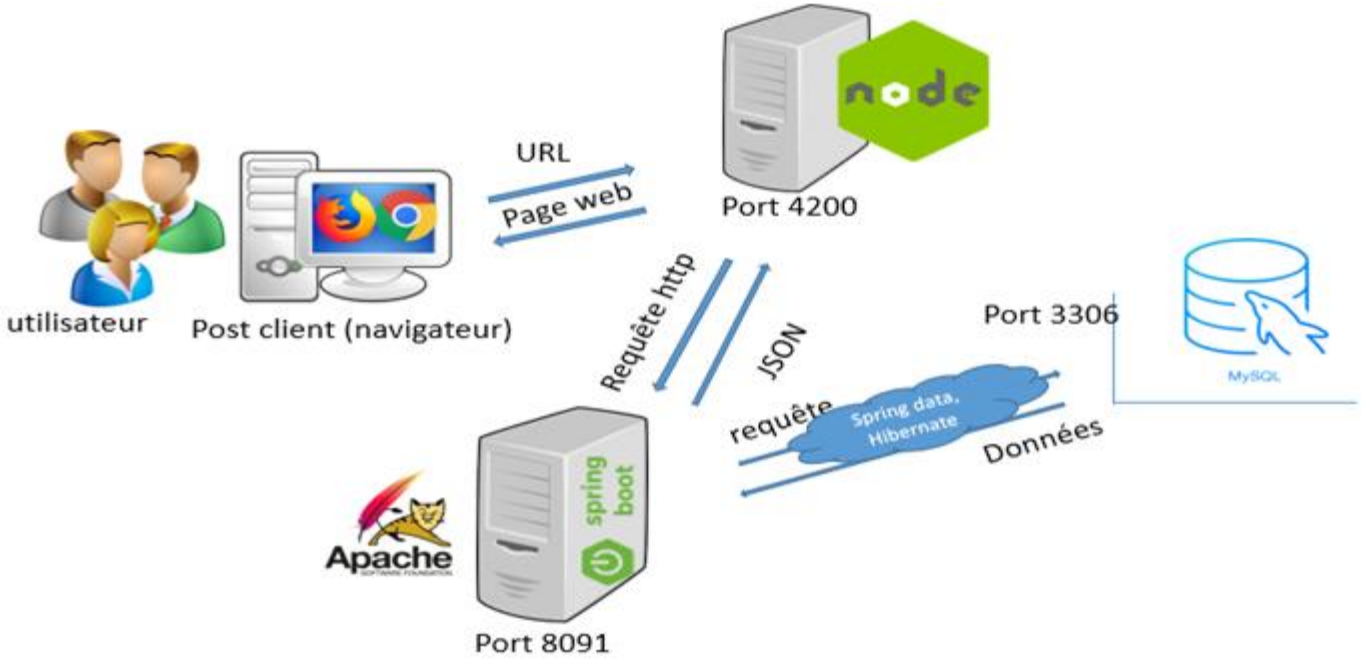
Architecture Physique - 3-Tiers

- Une application **3-Tiers** est typiquement une application Web :
  - Niveau **Présentation** : IHM (Navigateur sur la machine de l'utilisateur)
  - Niveau **Traitement** : Un serveur web (Tomcat, …) qui contient le WAR de notre application.
  - Niveau **Base de données** : Un serveur de BD qui stocke les données de notre application.

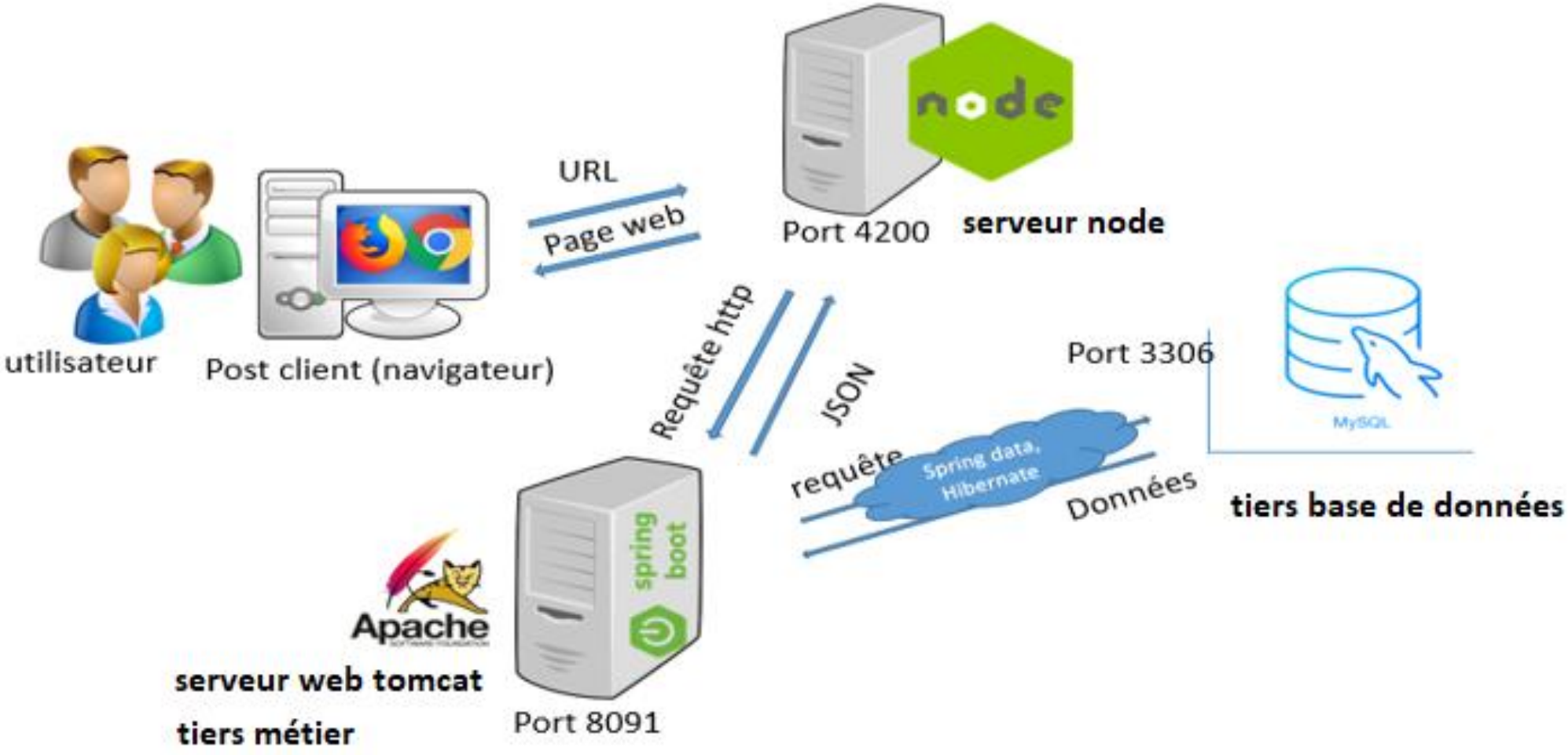


Architecture Physique - N-Tiers

- L'architecture N tiers assure un équilibre de charge entre le client et le serveur par l'introduction de nouvelles couches.
- Voici une architecture 4-Tiers d'une application web développée par un étudiant Esprit pendant son projet de fin d'étude (GUI – Angular sur le Serveur NodeJS – Spring Boot (Serveur Web Tomcat embarqué) – Serveur de base de données MySQL) :

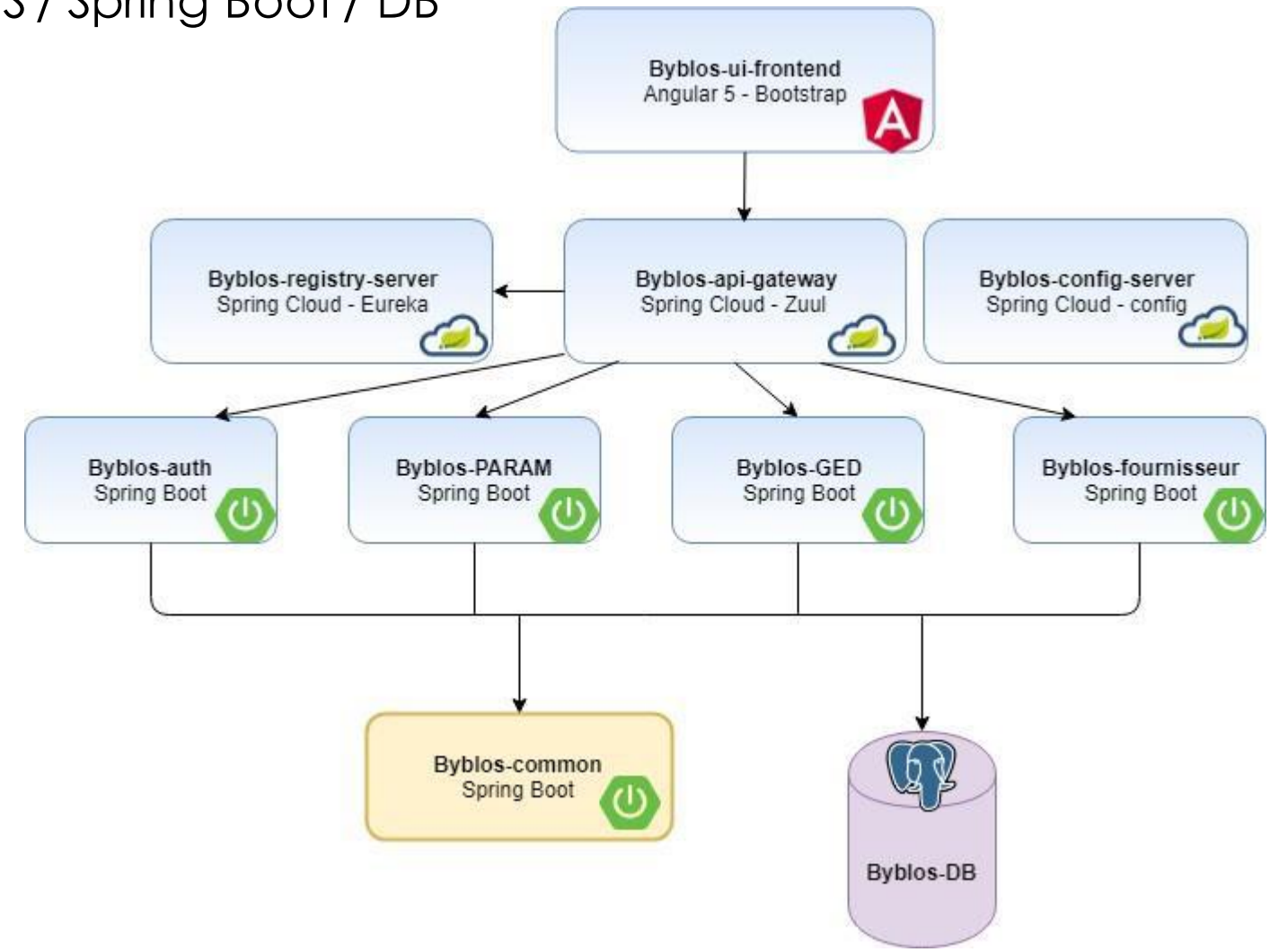


Architecture Physique - N-Tiers

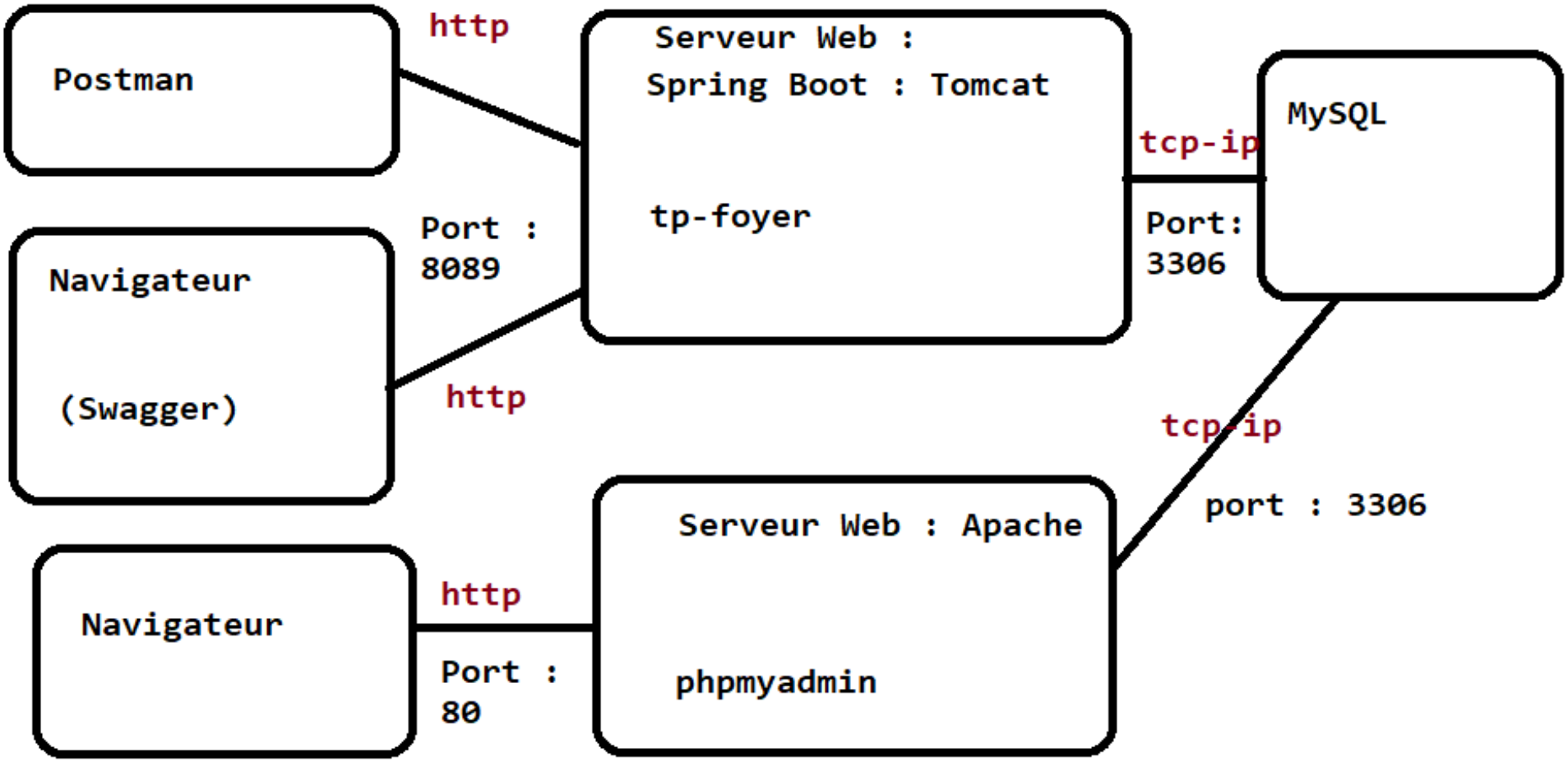


Architecture Physique - N-Tiers

- Voici une architecture n-Tiers, **en Micro-Servcies**, d'une application web développée par un étudiant Esprit pendant son projet de fin d'étude : GUI / NodeJS / Spring Boot / DB



Architecture Physique de notre Application



Architecture logique

- Une application typique utilisant Spring est généralement structurée en couches (5 couches : **Web / Controller / Service / Repository / Entité**) au lieu de l'architecture logique simplifiée en 3 couches (Présentation / Service / Accès aux Données) :

  - Couche **Web** (non fournie par Spring)
  - Couche **Contrôleur**

  ⇒ Couche **Présentation** (Web + Contrôleur)

  - Couche **Service** : interface métier avec mise en œuvre de certaines fonctionnalités.
  - Couche **Repository** (Recherche et Persistance des objets, CRUD)

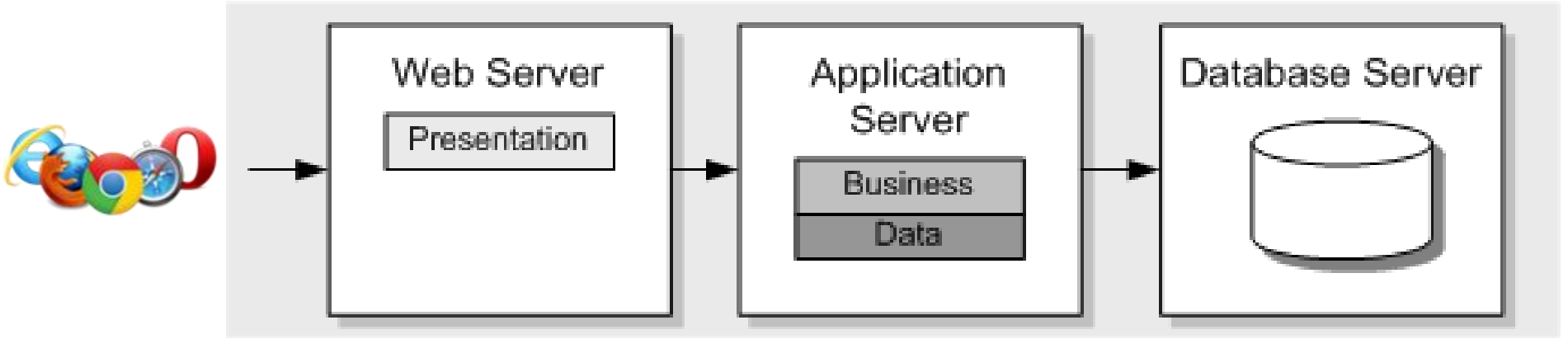
  ⇒ Couche **Métier** (Service + Repository)

  - Couche **Entités** (Modèle de données)

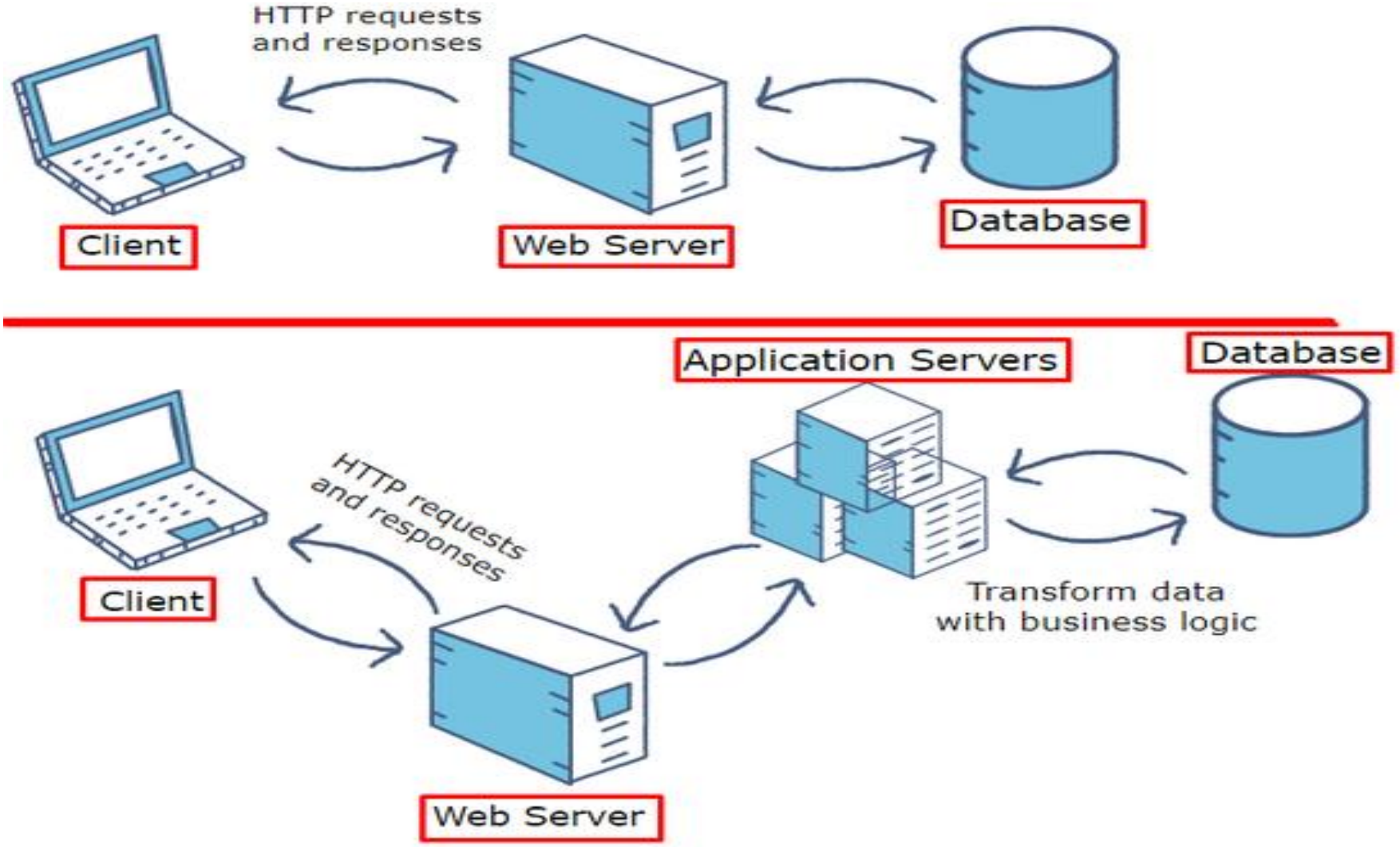
- **Spring est un Framework utilisé pour créer et injecter les objets requis pour communiquer entre les différentes couches.**

Serveur Web vs Serveur d'Application

Serveur Web	Serveur d'application JavaEE Serveur web + container
Héberge que la couche présentation et l'expose qu'à travers le protocole HTTP(S)	Héberge la logique métier et peut aussi héberger la couche présentation (supporte différents protocoles : HTTP, JNDI, ...).
Ne peut pas inclure un EJB Container.	Doit inclure un EJB Container.
lightweight	Relativement gourmand en ressources (CPU, RAM et Disk).
Exp : Apache HTTP Server, Tomcat, Jetty	Exp : Wildfly, WebSphere ...

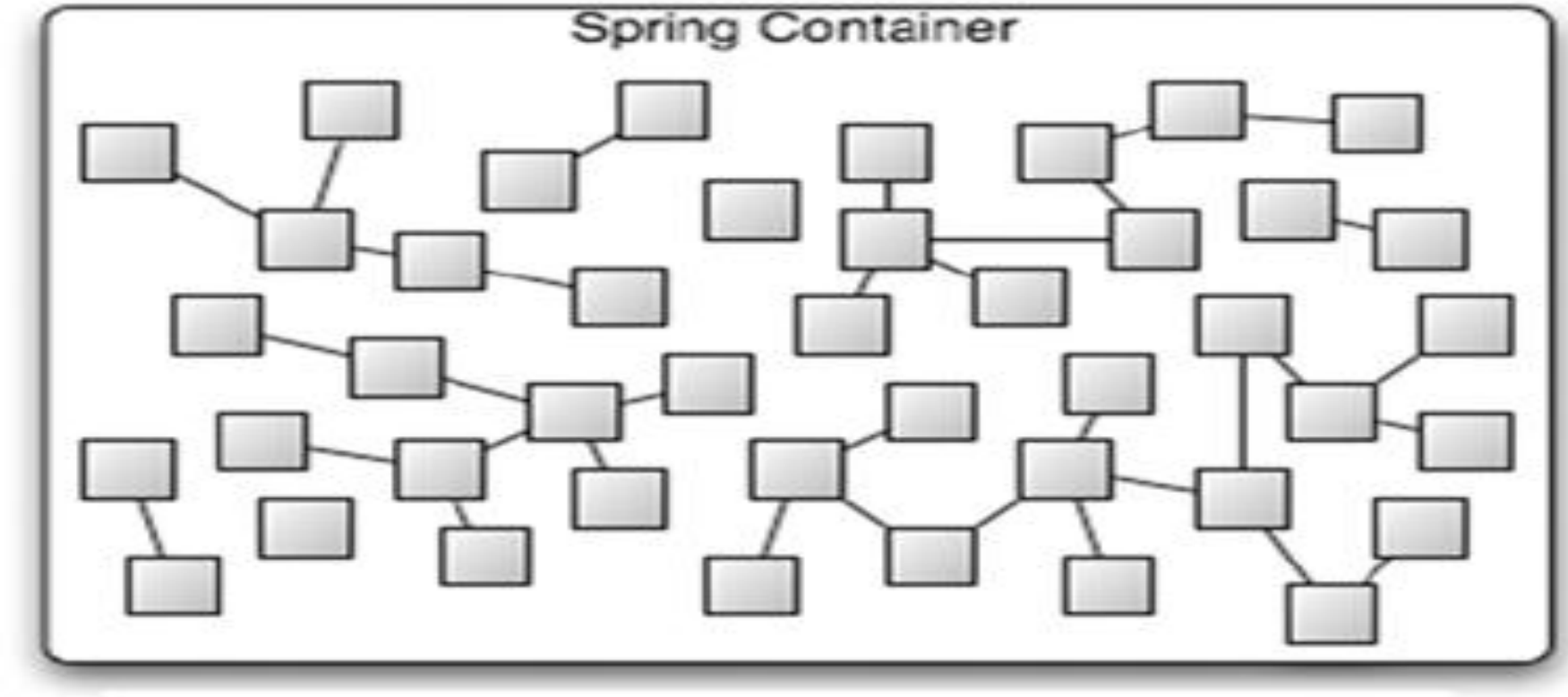


Serveur Web vs Serveur d'Application



Serveur Web vs Serveur d'Application

Spring IOC Container



Dans une application Spring, les objets sont créés, sont liés ensembles et communiquent dans le Spring IOC Container.

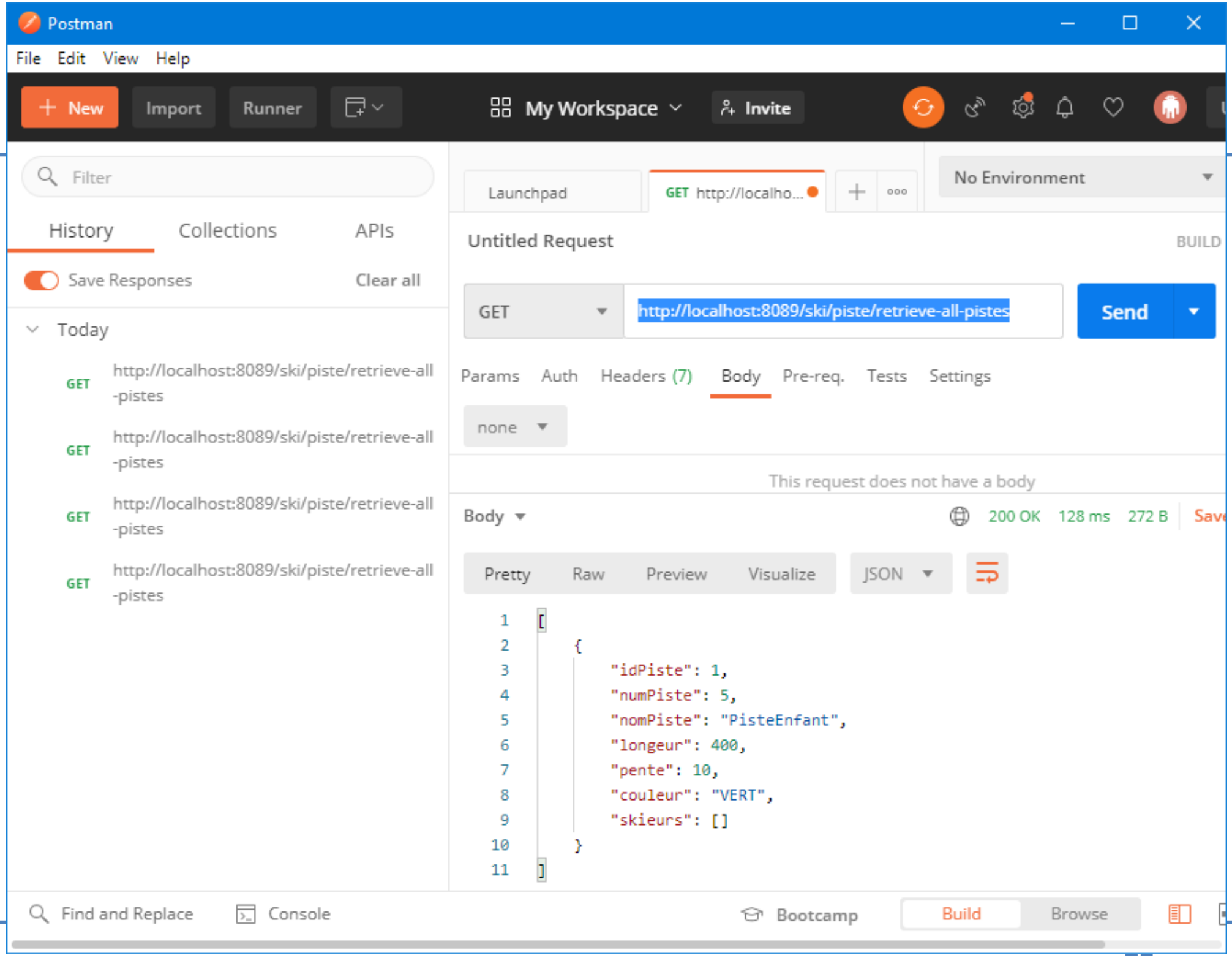
Spring MVC + Postman

Postman

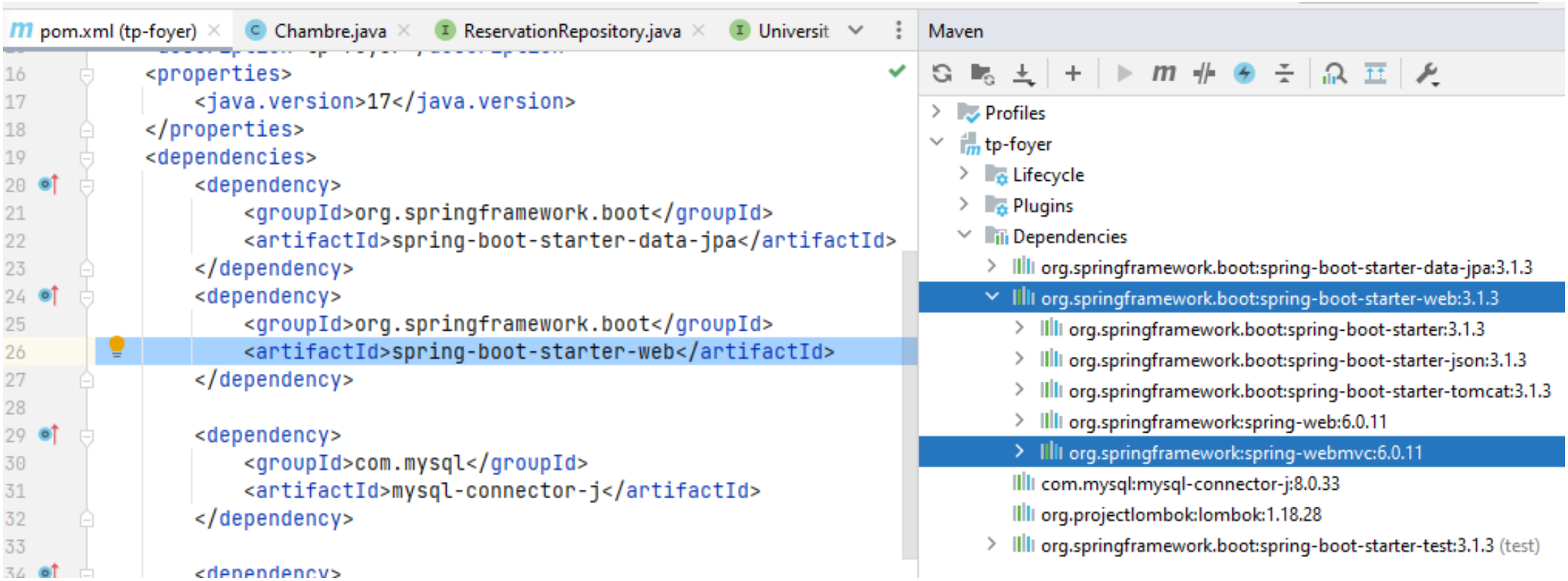
- Parmi les nombreuses solutions pour interroger ou tester les web services et les API, Postman propose de nombreuses fonctionnalités, une prise en main rapide et une interface graphique agréable.

- Postman permet de construire et d'exécuter des requêtes HTTP, de les stocker dans un historique afin de pouvoir les rejouer.




Postman
File Edit View Help
New
Import
Runner
My Workspace
Invite
Filter
History
Collections
APIs
Save Responses
Clear all
Today
GET http://localhost:8089/ski/piste/retrieve-all-pistes
GET http://localhost:8089/ski/piste/retrieve-all-pistes
GET http://localhost:8089/ski/piste/retrieve-all-pistes
GET http://localhost:8089/ski/piste/retrieve-all-pistes
Launchpad
GET http://localho...
No Environment
Untitled Request
BUILD
GET
http://localhost:8089/ski/piste/retrieve-all-pistes
Send
Params Auth Headers (7) Body Pre-req. Tests Settings
none
This request does not have a body
Body
200 OK 128 ms 272 B
Pretty Raw Preview Visualize JSON
[
{
"idPiste": 1,
"numPiste": 5,
"nomPiste": "PisteEnfant",
"longeur": 400,
"pente": 10,
"couleur": "VERT",
"skieurs": []
}
]
Find and Replace
Console
Bootcamp
Build
Browse

Dépendance web



- Le starter web permet d'ajouter toutes les dépendances liées à la partie web notamment celles liées à Spring MVC et l'exposition des web services.

Url de notre Application Web

- Dans le fichier de properties application.properties, ajouter les lignes suivantes, pour définir l'url de notre application :

```
#Server configuration
server.port=8089
server.servlet.context-path=/foyer
```

TP - Spring : Boot – Core – Data JPA – MVC (REST)

- Nous allons exposer des Web Service REST :
- Nous allons reprendre le même projet « tp-foyer » et exposer les méthodes (CRUD) avec des Web Service REST.
- Ces Web Services seront testé avec **Postman et avec Spring Doc (Swagger)** (voir support de cours suivant pour Swagger).

TP - Spring : Boot – Core – Data JPA – MVC (REST)

- Installation de Postman :

- Télécharger et à installer Postman.



TP - Spring : Boot – Core – Data JPA – MVC (REST)

- Vérifier que le fichier de properties contient les propriétés nécessaires (web, base de données, …) :

```
# DB Configuration :
spring.datasource.url=jdbc:mysql://localhost:3306/tpfoyerdb?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

# Server configuration :
server.port=8089
server.servlet.context-path=/tpfoyer
```

TP - Spring : Boot – Core – Data JPA – MVC (REST)

- Nous allons créer le Service ChambreServiceImpl et l'injecter dans le contrôleur.

- Créer les package **tn.esprit.tpfoyer.service**

- Créer **l'interface IChambreService**
- Créer le bean Spring **ChambreServiceImpl** annoté **@Service** qui implémente l'interface **IChambreService**

- Créer le CRUD dans le Service (voir pages suivantes) :

TP - Spring : Boot – Core – Data JPA – MVC (REST)

```
package tn.esprit.tpfoyer.service;

import tn.esprit.tpfoyer.entity.Chambre;

import java.util.List;

public interface IChambreService {

    public List<Chambre> retrieveAllChambres();
    public Chambre retrieveChambre(Long chambreId);
    public Chambre addChambre(Chambre c);
    public void removeChambre(Long chambreId);
    public Chambre modifyChambre(Chambre chambre);

    // Here we will add later methods calling keywords and methods calling JPQL

}
```

TP - Spring : Boot – Core – Data JPA – MVC (REST)

```
@Service
@AllArgsConstructor
public class ChambreServiceImpl implements IChambreService {

    ChambreRepository chambreRepository;

    public List<Chambre> retrieveAllChambres() {
        return chambreRepository.findAll();
    }
    public Chambre retrieveChambre(Long chambreId) {
        return chambreRepository.findById(chambreId).get();
    }
    public Chambre addChambre(Chambre c) {
        return chambreRepository.save(c);
    }
    public void removeChambre(Long chambreId) {
        chambreRepository.deleteById(chambreId);
    }
    public Chambre modifyChambre(Chambre chambre) {
        return chambreRepository.save(chambre);
    }
}
```

TP - Spring : Boot – Core – Data JPA – MVC (REST)

Maintenant nous allons créer le rest controlleur :

- Créer les package **tn.esprit.tpfoyer.control**
- Créer le bean Spring **ChambreRestController** annoté **@RestController**
- Créer les méthodes nécessaires pour exposer le CRUD (voir pages suivantes).

TP - Spring : Boot – Core – Data JPA – MVC (REST)

```
@RestController
@AllArgsConstructor
@RequestMapping("/chambre")
public class ChambreRestController {

    IChambreService chambreService;

    // http://localhost:8089/tpfoyer/chambre/retrieve-all-chambres
    @GetMapping("/retrieve-all-chambres")
    public List<Chambre> getChambres() {
        List<Chambre> listChambres = chambreService.retrieveAllChambres();
        return listChambres;
    }

    // http://localhost:8089/tpfoyer/chambre/retrieve-chambre/8
    @GetMapping("/retrieve-chambre/{chambre-id}")
    public Chambre retrieveChambre(@PathVariable("chambre-id") Long chId) {
        Chambre chambre = chambreService.retrieveChambre(chId);
        return chambre;
    }
```

(Suite de la classe page suivante) :

TP - Spring : Boot – Core – Data JPA – MVC (REST)

```
    // http://localhost:8089/tpfoyer/chambre/add-chambre
    @PostMapping("/add-chambre")
    public Chambre addChambre(@RequestBody Chambre c) {
        Chambre chambre = chambreService.addChambre(c);
        return chambre;
    }

    // http://localhost:8089/tpfoyer/chambre/remove-chambre/{chambre-id}
    @DeleteMapping("/remove-chambre/{chambre-id}")
    public void removeChambre(@PathVariable("chambre-id") Long chId) {
        chambreService.removeChambre(chId);
    }

    // http://localhost:8089/tpfoyer/chambre/modify-chambre
    @PutMapping("/modify-chambre")
    public Chambre modifyChambre(@RequestBody Chambre c) {
        Chambre chambre = chambreService.modifyChambre(c);
        return chambre;
    }

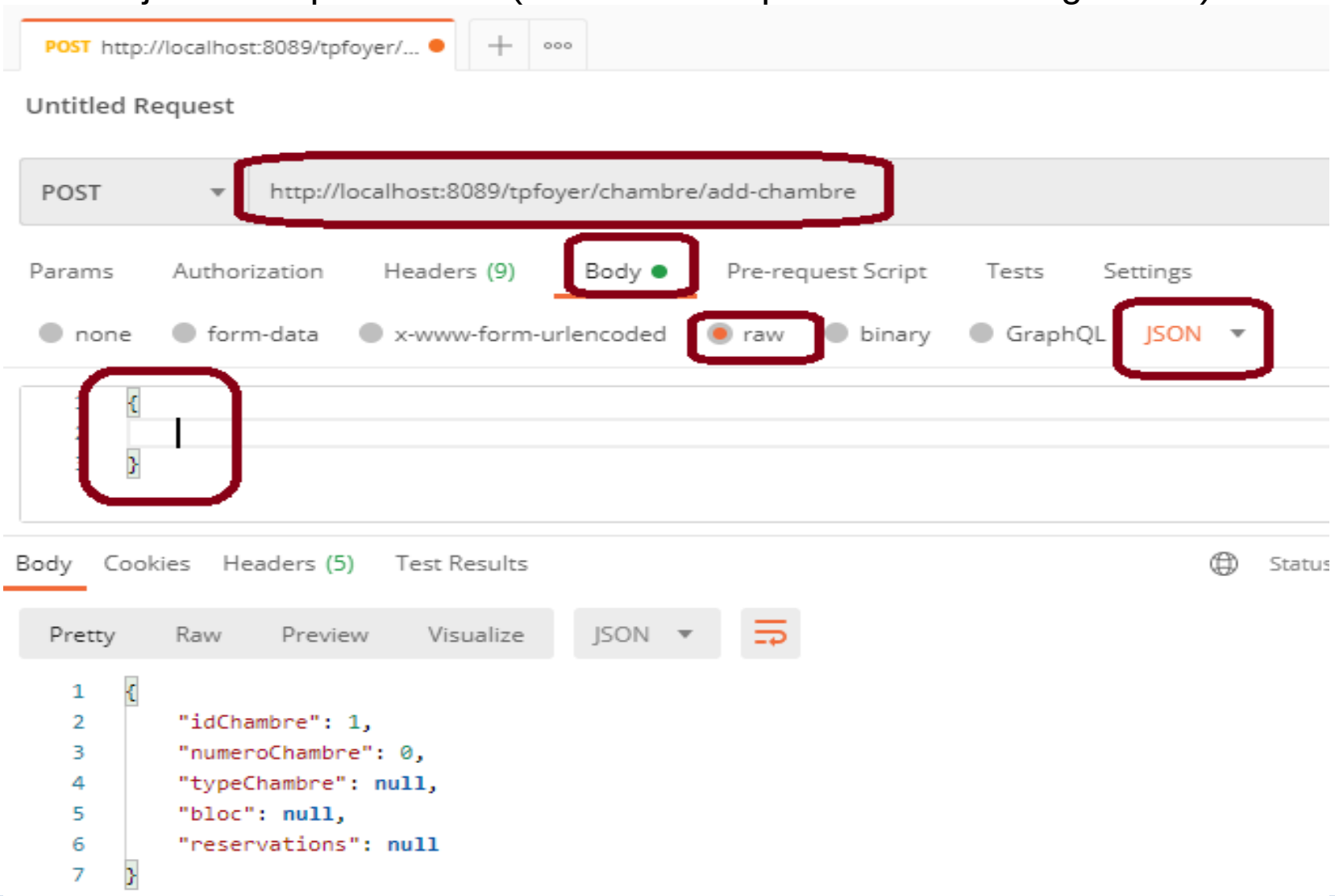
}
```

TP - Spring : Boot – Core – Data JPA – MVC (REST)

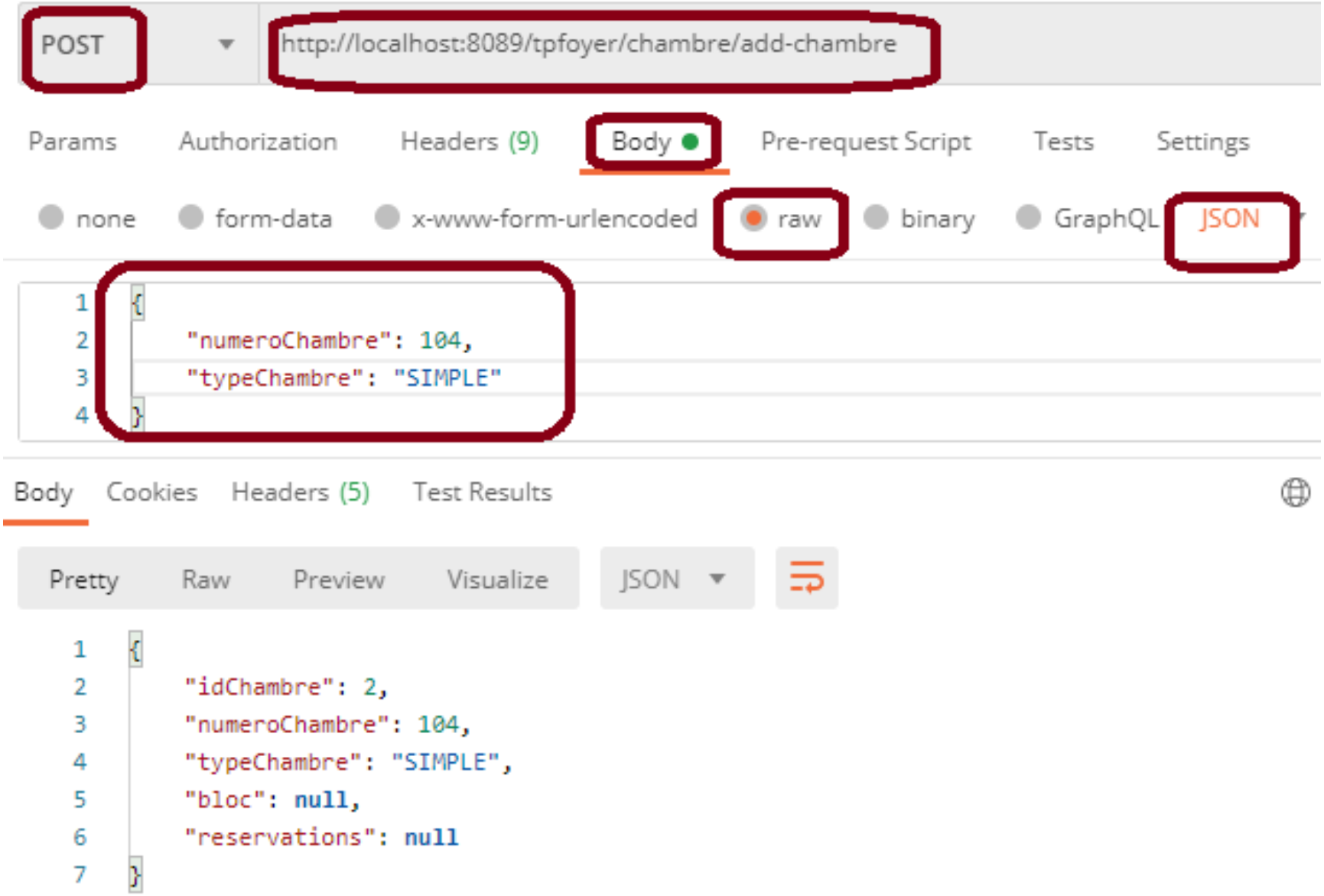
- Tester l'ensemble des méthodes exposées, avec Postman
- La séance prochaine, les web services exposés seront testés avec Spring Doc (Swagger).

Postman

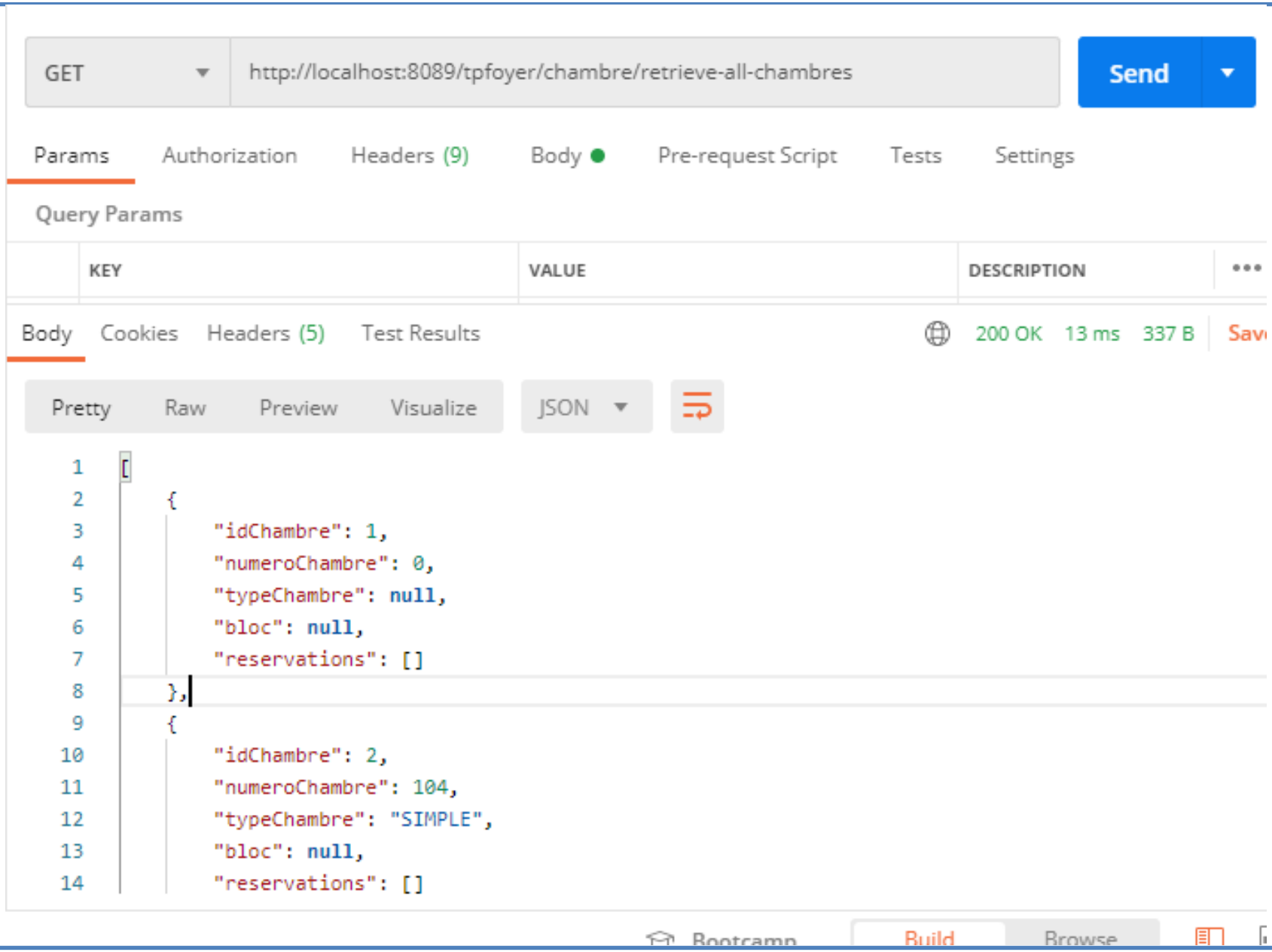
Pour avoir la structure de l'objet JSON chambre, commencer par créer une chambre sans attributs, puis utiliser cet objet JSON par la suite (enlever la clé primaire car auto générée) :



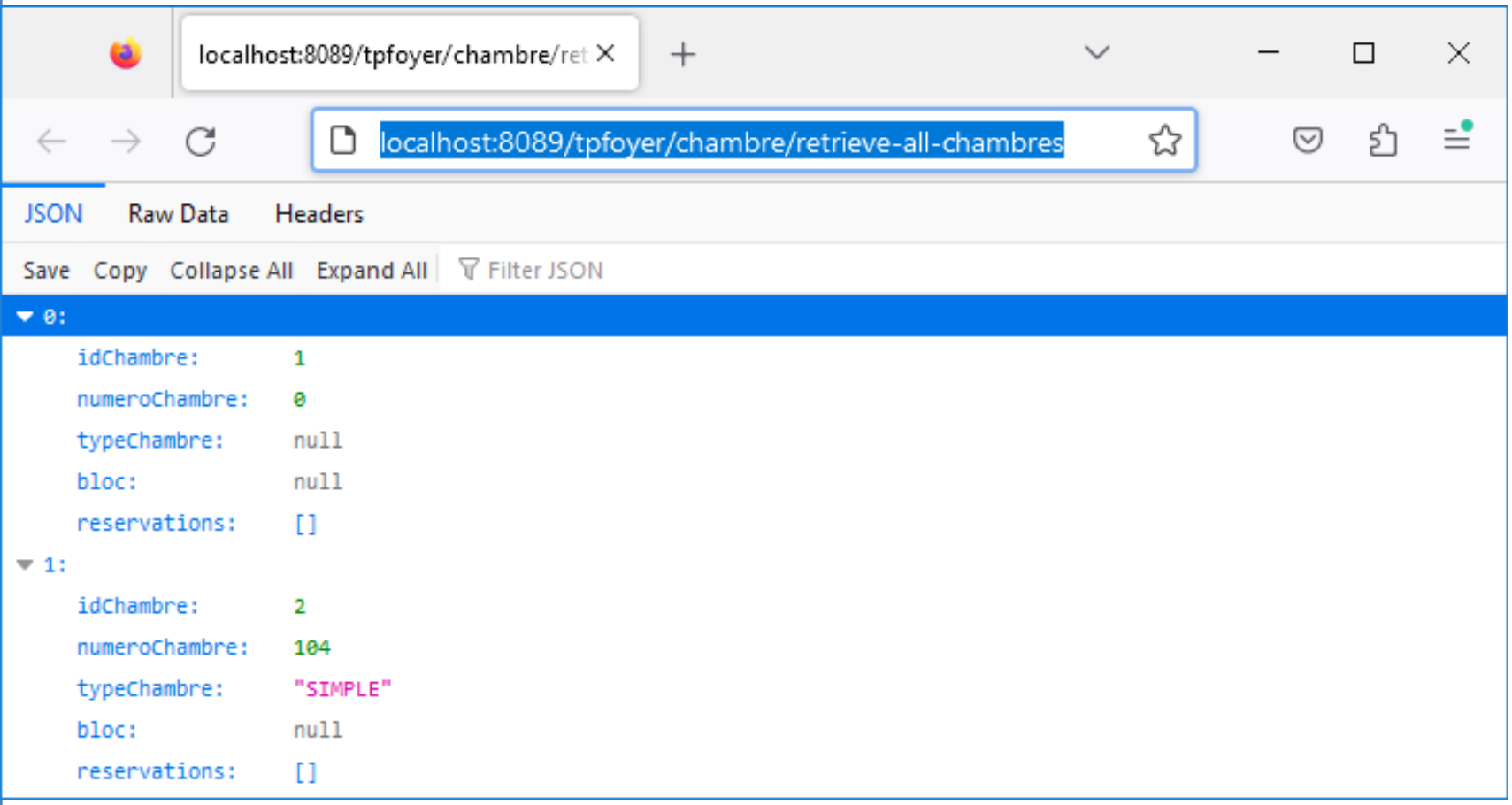
TP - Spring : Boot – Core – Data JPA – MVC (REST)

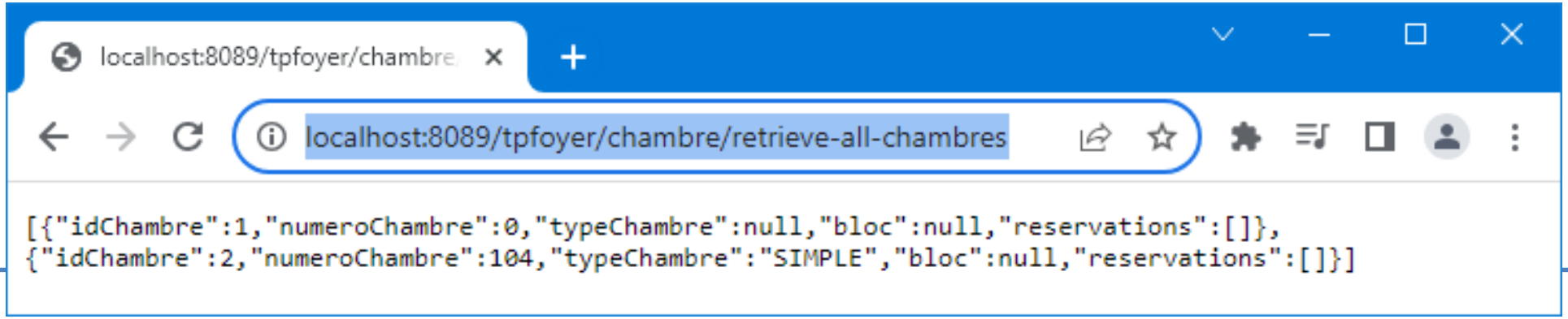


Liste des Chambres(Navigateur + Postman)



Liste des Chambres(Navigateur + Postman)





La séance prochaine :

- Les web services exposés (liés à l'entité Chambre) seront testés avec Spring Doc (Swagger).
- Les web services liés aux autres entités seront exposés et testés avec Postman et Spring Doc.